DOCUMENTAÇÃO TÉCNICA

PokéDex Web

Aplicação Laravel 13 com Integração PokeAPI

Projeto:	PokéDex Web
Tipo:	Aplicação Web Full-Stack
Framework:	Laravel 13.6 (PHP 8.3)
Banco de dados:	MySQL — PokemonDB
Categoria:	Documentação Acadêmica e Técnica

PHP 8.3 · Laravel 13.6 · MySQL · Blade · Tailwind CSS v4 · PokeAPI

Índice

1. Visão Geral do Projeto
   - 1.1 Descrição
   - 1.2 Funcionalidades Principais
2. Stack Tecnológica
3. Arquitetura MVC
   - 3.1 Conceito de MVC
   - 3.2 Aplicação no Projeto
4. Camada de Rotas
5. Controllers
   - 5.1 PokemonController
   - 5.2 MeuPokemonController
6. Camada de Modelos (Eloquent ORM)
   - 6.1 Eloquent ORM e Mass Assignment
   - 6.2 Model MeuPokemon
7. Migrations e Estrutura do Banco
8. Seeders
9. Camada de Visualização (Blade)
   - 9.1 Como o Blade Compila Templates
   - 9.2 Views do Projeto
10. Conceitos Fundamentais
    - 10.1 Route Model Binding

10.2 Proteção CSRF

10.3 Storage e Symlink Público

10.4 Unificação de Fontes de Dados

11. Boas Práticas e Armadilhas

12. Conclusão

1. Visão Geral do Projeto

1.1 Descrição

O **PokéDex Web** é uma aplicação web desenvolvida em **Laravel 13** cujo objetivo é unir o consumo de uma API pública (a PokeAPI) com a manutenção de uma coleção pessoal de Pokémons customizados, persistidos em banco de dados local. O sistema foi pensado como um exercício didático e prático, contemplando os principais pilares do desenvolvimento web moderno em PHP: roteamento, controladores, modelos via ORM, migrations, seeders, templates Blade e gerenciamento de arquivos via Storage.

A aplicação demonstra na prática como o Laravel padroniza o ciclo de vida de uma requisição HTTP — da entrada da rota até a resposta renderizada pelo Blade — e como integrar de forma transparente fontes de dados heterogêneas (uma API REST externa e uma tabela MySQL local) em um único formato consumível pela camada de apresentação.

1.2 Funcionalidades Principais

- **Busca de Pokémon:** o usuário digita o nome e o sistema procura primeiro no banco local; se não encontrar, consulta a PokeAPI.
- **Pokémon aleatório:** sorteio entre os IDs 1 e 1025 com chamada à PokeAPI.
- **Cadastro de Pokémon customizado:** formulário com upload de imagem, escolha de tipos, atributos numéricos e habilidades.
- **Listagem da coleção pessoal:** grid renderizado na página principal.
- **Exclusão de Pokémon customizado:** com remoção da imagem associada do disco.

2. Stack Tecnológica

A tabela a seguir resume todas as tecnologias utilizadas e o papel de cada uma no projeto:

Tecnologia	Versão / Origem	Função no Projeto
PHP	8.3	Linguagem de back-end
Laravel	13.6	Framework MVC principal
MySQL	banco `PokemonTB`	Persistência da coleção customizada
Blade	nativo do Laravel	Template engine das views
Tailwind CSS	v4 via CDN	Estilização utilitária
Google Fonts	Press Start 2P + Inter	Tipografia (estética retrô e leitura)
PokeAPI	pokeapi.co/api/v2/pokemon/{name}	Fonte externa de dados de Pokémon
Laravel Storage	disco `public` + symlink	Upload e exposição de imagens

3. Arquitetura MVC

3.1 Conceito de MVC

MVC é a sigla para **Model–View–Controller**, um padrão arquitetural que separa a aplicação em três camadas com responsabilidades distintas. O **Model** representa os dados e a lógica de negócio; a **View** cuida da apresentação ao usuário; e o **Controller** faz a ponte entre os dois, recebendo a requisição, orquestrando a lógica e devolvendo uma resposta.

> **Por que MVC importa?** Essa separação torna o código mais fácil de manter, testar e escalar. Mudar a aparência da tela não exige mexer nas regras de negócio, e vice-versa.

3.2 Aplicação no Projeto

Camada	No projeto	Localização
Model	MeuPokemon (Eloquent)	`app/Models/MeuPokemon.php`
View	welcome, pokemon/create	`resources/views/`
Controller	PokemonController, MeuPokemonController	`app/Http/Controllers/`
Roteador	Mapeia URL → Controller	`routes/web.php`

4. Camada de Rotas

As rotas são o ponto de entrada da aplicação: elas definem qual URL aciona qual método de qual controller. Toda rota possui um *nome* (atributo `name()`) que permite referenciá-la nas views via `route('nome')`, evitando o uso de URLs hardcoded.

Método	URL	Controller@método	Nome da rota
GET	`/`	PokemonController@index	home
POST	`/pokemon/buscar`	PokemonController@search	pokemon.search
GET	`/pokemon/aleatorio`	PokemonController@random	pokemon.random
GET	`/meu-pokemon/criar`	MeuPokemonController@create	meu-pokemon.create
POST	`/meu-pokemon`	MeuPokemonController@store	meu-pokemon.store
DELETE	`/meu-pokemon/{meuPokemon}`	MeuPokemonController@destroy	meu-pokemon.destroy

routes/web.php

```
use App\Http\Controllers\PokemonController;
use App\Http\Controllers\MeuPokemonController;

Route::get('/', [PokemonController::class, 'index'])->name('home');
Route::post('/pokemon/buscar', [PokemonController::class, 'search'])->name('pokemon.search');
Route::get('/pokemon/aleatorio', [PokemonController::class, 'random'])->name('pokemon.random');

Route::get('/meu-pokemon/criar', [MeuPokemonController::class, 'create'])->name('meu-pokemon.create');
Route::post('/meu-pokemon', [MeuPokemonController::class, 'store'])->name('meu-pokemon.store');
Route::delete('/meu-pokemon/{meuPokemon}', [MeuPokemonController::class, 'destroy'])->name('meu-pokemon
```

5. Controllers

Os controllers concentram a lógica de orquestração: recebem a `Request`, validam dados, acionam Models e/ou serviços externos e retornam uma View ou um redirecionamento.

5.1 PokemonController

Responsável por toda a interação com a PokeAPI e pela busca unificada (local + externa).

Método	Função
`index()`	Carrega todos os registros de `MeuPokemon` e devolve a view `welcome`.
`search(Request)`	Procura primeiro no banco local com `whereRaw LOWER`; se não achar, consulta a PokeAPI.
`random()`	Gera um ID aleatório entre 1 e 1025 e busca o Pokémon correspondente na PokeAPI.
`format(array)`	Normaliza o JSON da PokeAPI para um array padrão consumido pela view.
`formatLocal(MeuPokemon)`	Normaliza um registro do banco para o mesmo formato (altura×10 em decímetros, peso×10 em hectogramas, `Storage::url()` para imagem).

PokemonController.php (trecho)

```php
public function search(Request $request)
{
    $nome = strtolower(trim($request->input('nome')));

    // 1) busca primeiro no banco local
    $local = MeuPokemon::whereRaw('LOWER(nome) = ?', [$nome])->first();
    if ($local) {
        return view('welcome', [
            'pokemon'      => $this->formatLocal($local),
            'meusPokemons' => MeuPokemon::all(),
        ]);
    }

    // 2) fallback: PokeAPI
    $response = Http::get("https://pokeapi.co/api/v2/pokemon/{$nome}");
    if ($response->successful()) {
        return view('welcome', [
            'pokemon'      => $this->format($response->json()),
            'meusPokemons' => MeuPokemon::all(),
        ]);
    }

    return back()->with('erro', 'Pokémon não encontrado.');
}
```

5.2 MeuPokemonController

Cuida do CRUD da coleção pessoal.

Método	Função
`create()`	Passa `$tiposDisponiveis` e `$tiposLabels` para a view do formulário.
`store(Request)`	Valida → monta a string de tipos → faz o upload da imagem → `MeuPokemon::create()`.
`destroy(MeuPokemon)`	Recebe o model via **Route Model Binding** → `Storage::delete()` → `$model->delete()`.

MeuPokemonController.php (trecho — store)

```php
public function store(Request $request)
{
    $dados = $request->validate([
        'nome'    => 'required|string|max:255',
        'tipos'   => 'required|array|min:1',
        'imagem'  => 'nullable|image|max:2048',
        'altura'  => 'required|numeric|min:0',
        'peso'    => 'required|numeric|min:0',
        // demais stats...
    ]);

    $dados['tipos'] = implode(',', $request->input('tipos'));

    if ($request->hasFile('imagem')) {
        $dados['imagem'] = $request->file('imagem')
            ->store('pokemon-images', 'public');
    }

    MeuPokemon::create($dados);
    return redirect()->route('home')->with('sucesso', 'Pokémon criado!');
}
```

6. Camada de Modelos (Eloquent ORM)

6.1 Eloquent ORM e Mass Assignment

ORM (Object-Relational Mapping) é uma técnica que mapeia tabelas de banco relacional para classes PHP e linhas para objetos. O **Eloquent** é o ORM nativo do Laravel: cada Model representa uma tabela, e métodos como `find()`, `where()`, `create()` ou `delete()` traduzem-se internamente em SQL.

> **Mass Assignment Protection.** Quando se chama `Model::create($array)`, o Eloquent atribui em massa todas as colunas presentes no array. Sem proteção, um atacante poderia injetar campos como `is_admin` em um formulário. Por isso o Laravel exige a declaração explícita do array `$fillable`: somente colunas listadas ali podem ser preenchidas em massa.

6.2 Model MeuPokemon

app/Models/MeuPokemon.php

```
class MeuPokemon extends Model
{
    protected $table = 'meus_pokemons';

    protected $fillable = [
        'nome', 'imagem', 'tipos',
        'altura', 'peso',
        'hp', 'ataque', 'defesa',
        'ataque_especial', 'defesa_especial',
        'velocidade', 'habilidades',
    ];

    public function tiposArray(): array
    {
        return explode(',', $this->tipos);
    }

    public function stats(): array
    {
        return [
            ['name' => 'hp',              'label' => 'HP',         'value' => $this->hp],
            ['name' => 'ataque',          'label' => 'Ataque',     'value' => $this->ataque],
            ['name' => 'defesa',          'label' => 'Defesa',     'value' => $this->defesa],
            ['name' => 'ataque_especial', 'label' => 'Atq. Esp.',  'value' => $this->ataque_especial],
            ['name' => 'defesa_especial', 'label' => 'Def. Esp.',  'value' => $this->defesa_especial],
            ['name' => 'velocidade',      'label' => 'Velocidade', 'value' => $this->velocidade],
        ];
    }
}
```

Os métodos `tiposArray()` e `stats()` são *accessors* de conveniência: encapsulam a lógica de formatação dentro do próprio Model, mantendo as views simples.

7. Migrations e Estrutura do Banco

Uma **migration** é um arquivo PHP que descreve uma alteração de esquema (criar tabela, adicionar coluna, criar índice etc.). Em vez de criar a tabela manualmente no MySQL, define-se a estrutura em código versionado.

Por que usar migrations?

- **Versionamento:** entram no Git junto com o código.
- **Reprodutibilidade:** qualquer dev roda `php artisan migrate` e fica com o mesmo schema.
- **Reversibilidade:** o método `down()` permite desfazer.
- **Independência de SGBD:** a mesma migration roda em MySQL, PostgreSQL, SQLite etc.

Estrutura da tabela `meus_pokemons`

Coluna	Tipo	Detalhes
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
nome	VARCHAR	Obrigatório
imagem	VARCHAR	Nullable (caminho relativo)
tipos	VARCHAR	Default `'normal'` (lista separada por vírgula)
altura	DECIMAL(5,2)	Em metros
peso	DECIMAL(6,2)	Em quilogramas
hp	SMALLINT UNSIGNED	Default 45
ataque	SMALLINT UNSIGNED	Default 45
defesa	SMALLINT UNSIGNED	Default 45
ataque_especial	SMALLINT UNSIGNED	Default 45
defesa_especial	SMALLINT UNSIGNED	Default 45
velocidade	SMALLINT UNSIGNED	Default 45
habilidades	VARCHAR	Nullable
created_at / updated_at	TIMESTAMP	Gerenciados automaticamente pelo Eloquent

database/migrations/...create_meus_pokemons_table.php

```
Schema::create('meus_pokemons', function (Blueprint $table) {
    $table->id();
    $table->string('nome');
    $table->string('imagem')->nullable();
    $table->string('tipos')->default('normal');
    $table->decimal('altura', 5, 2);
    $table->decimal('peso', 6, 2);
    $table->unsignedSmallInteger('hp')->default(45);
    $table->unsignedSmallInteger('ataque')->default(45);
    $table->unsignedSmallInteger('defesa')->default(45);
    $table->unsignedSmallInteger('ataque_especial')->default(45);
    $table->unsignedSmallInteger('defesa_especial')->default(45);
    $table->unsignedSmallInteger('velocidade')->default(45);
    $table->string('habilidades')->nullable();
    $table->timestamps();
});
```

8. Seeders

Seeder é uma classe que popula o banco com dados de exemplo ou de configuração. É útil em ambientes de desenvolvimento e teste, onde se precisa de uma base previsível para validar funcionalidades.

No projeto, `MeusPokemonsSeeder` primeiro executa um `truncate()` para limpar a tabela e depois insere três Pokémons customizados de demonstração:

Nome	Tipos	HP	Atk	Def	SpAtk	SpDef	Spd
Aethelfin	water, rock	80	65	95	80	70	50
Foliagrove	grass, ground	75	90	85	55	70	45
Galehawk	flying, electric	70	95	60	75	65	110

database/seeders/MeusPokemonsSeeder.php

```
public function run(): void
{
    MeuPokemon::truncate();

    MeuPokemon::create([
        'nome' => 'Aethelfin',
        'tipos' => 'water,rock',
        'altura' => 1.40, 'peso' => 38.5,
        'hp' => 80, 'ataque' => 65, 'defesa' => 95,
        'ataque_especial' => 80, 'defesa_especial' => 70, 'velocidade' => 50,
    ]);
    // ... Foliagrove e Galehawk
}
```

Execução: `php artisan db:seed --class=MeusPokemonsSeeder`

9. Camada de Visualização (Blade)

9.1 Como o Blade Compila Templates

O **Blade** é um template engine que estende o HTML com diretivas como `@if`, `@foreach`, `@csrf` e interpolação `{{ $var }}`. Quando uma view é renderizada pela primeira vez, o Blade **compila o arquivo .blade.php em PHP puro** e armazena o resultado em `storage/framework/views/`. Nas requisições seguintes, o arquivo compilado é reutilizado enquanto o original não for alterado — o que torna o Blade rápido em produção.

> **Atenção ao parser do Blade.** Nunca escreva diretivas `@` dentro de comentários Blade `{{-- --}}`. O parser *ainda* processa essas diretivas no comentário, gerando erros silenciosos ou comportamentos inesperados. Se precisar comentar trechos com diretivas, use comentários HTML (`<!-- -->`) ou remova a arroba.

9.2 Views do Projeto

welcome.blade.php

Página principal. Contém o cabeçalho, a barra de busca, o card do Pokémon encontrado (com cores dinâmicas baseadas no tipo) e o grid de "Meus Pokémons" com botão de exclusão. Recebe duas variáveis principais: `$pokemon` (resultado da busca, pode ser nulo) e `$meusPokemons` (Collection do Eloquent).

pokemon/create.blade.php

Formulário de criação de Pokémon customizado. Implementa três interatividades em JavaScript puro:

- **Preview de imagem:** via `FileReader API`, exibindo a imagem selecionada antes do envio.
- **Selects de tipo:** com cores específicas por tipo, sincronizadas com a paleta do card de exibição.
- **Barras de stat animadas:** escutam o evento `oninput` dos sliders e atualizam a largura da barra em tempo real.

Fluxo de Imagens

O upload da imagem segue o caminho a seguir:

```
Fluxo
```

```
1. Usuário envia <input type="file" name="imagem">
2. Controller chama: $request->file('imagem')->store('pokemon-images', 'public')
3. Arquivo gravado em: storage/app/public/pokemon-images/abc123.png
4. Symlink (php artisan storage:link) faz: public/storage -> storage/app/public
5. URL pública final: /storage/pokemon-images/abc123.png
6. Na view: Storage::url($model->imagem) gera o caminho correto
```

10. Conceitos Fundamentais

10.1 Route Model Binding

No método `destroy(MeuPokemon $meuPokemon)`, o controller recebe diretamente uma instância do Model em vez de receber um `$id` e precisar buscar manualmente. Isso é o **Route Model Binding**: o Laravel inspeciona a assinatura do método, extrai o valor do parâmetro da URL (`{meuPokemon}`), faz a query `MeuPokemon::findOrFail($valor)` automaticamente e injeta o objeto. Se não encontrar, devolve um 404 sem que o desenvolvedor escreva uma linha sequer.

Comparação

```
// SEM Route Model Binding (verboso, repetitivo)
public function destroy($id) {
    $pokemon = MeuPokemon::findOrFail($id);
    $pokemon->delete();
}

// COM Route Model Binding (limpo, expressivo)
public function destroy(MeuPokemon $meuPokemon) {
    $meuPokemon->delete();
}
```

10.2 Proteção CSRF

CSRF (Cross-Site Request Forgery) é um ataque em que um site malicioso induz o navegador da vítima — já autenticado em outra aplicação — a enviar requisições indesejadas a essa aplicação. O Laravel mitiga isso exigindo um **token CSRF** em todo formulário POST/PUT/DELETE. A diretiva `@csrf` no Blade insere um campo `<input type="hidden" name="_token">` com um valor associado à sessão; o middleware `VerifyCsrfToken` compara esse valor com o token da sessão antes de permitir a execução do controller.

Exemplo

```
<form method="POST" action="{{ route('meu-pokemon.store') }}">
    @csrf
    <input type="text" name="nome">
    <button type="submit">Criar</button>
</form>
```

Para o método DELETE, é necessário também usar `@method('DELETE')`, já que formulários HTML só suportam GET e POST nativamente.

10.3 Storage e Symlink Público

O Laravel separa, por design, os arquivos da aplicação dos arquivos públicos. Tudo que é enviado via upload vai para `storage/app/` — pasta **fora** do diretório servido pelo servidor web. Para tornar uma parte desses arquivos acessível pela web, usa-se o **disco public** (`storage/app/public`) e cria-se um link simbólico:

Terminal

```
php artisan storage:link
# cria: public/storage -> storage/app/public
```

A partir desse ponto, qualquer arquivo em `storage/app/public/qualquer-coisa` é acessível publicamente em `/storage/qualquer-coisa`. O método `Storage::url($caminho)` gera essa URL automaticamente.

10.4 Unificação de Fontes de Dados

Um dos pontos arquiteturais mais elegantes do projeto é como ele **unifica** dados vindos da PokeAPI e do banco local. Os dois métodos `format()` e `formatLocal()` produzem a **mesma estrutura de array**, de forma que a view `welcome.blade.php` não precisa saber a origem do Pokémon — ela apenas itera sobre as mesmas chaves.

Fonte	Altura	Peso	Imagem
PokeAPI	já em decímetros	já em hectogramas	URL direta de sprite
Banco local	metros × 10 → decímetros	kg × 10 → hectogramas	`Storage::url($imagem)`

Por que essa unificação importa? Em uma aplicação real, fontes de dados podem mudar (uma API pode ser substituída, uma tabela pode ser dividida). Quando a view depende apenas de um *contrato* (um formato fixo de array), trocar a fonte não exige reescrever templates. Esse princípio é uma forma simplificada do padrão *Adapter*.

11. Boas Práticas e Armadilhas

Boa prática	Por quê
Usar `$fillable` em todo Model	Bloqueia mass assignment indevido
Nomear todas as rotas	Permite mudar URLs sem quebrar views
Validar via `$request->validate()`	Falha rápida e mensagens automáticas
Usar Route Model Binding	Reduz boilerplate e dá 404 automático
Apagar arquivo no `destroy()`	Evita lixo em `storage/`
Comparar nomes em lower-case (`whereRaw LOWER`)	Busca insensível a caixa
Usar `Storage::url()`	Não hardcoda `/storage/` nas views

Armadilhas comuns

- **Esquecer `php artisan storage:link`:** imagens carregam mas retornam 404.
- **Esquecer `@csrf`:** formulário retorna erro 419 (Page Expired).
- **Diretiva Blade dentro de comentário `{{-- --}}`:** o parser ainda processa.
- **Cache de view obsoleto:** rodar `php artisan view:clear` ao mudar diretivas.
- **Migration sem `down()`:** impede rollback limpo.

12. Conclusão

O **PokéDex Web** é um projeto enxuto, mas que cobre, na prática, a quase totalidade dos conceitos estruturais do Laravel: roteamento nomeado, controllers com responsabilidade bem definida, Eloquent ORM com proteção contra mass assignment, migrations versionadas, seeders para dados de exemplo, integração com API externa via `Http` facade, upload de arquivos com Storage e symlink, Route Model Binding e proteção CSRF.

A decisão de unificar dois fontes de dados distintas (banco local e PokeAPI) em um único formato consumível pela view é um exercício pedagógico valioso: demonstra a separação entre *fonte* e *apresentação* e prepara o terreno para projetos onde múltiplos serviços precisam ser combinados de maneira transparente.

Como evolução natural, o projeto pode incorporar autenticação de usuários (Laravel Breeze ou Fortify), políticas de autorização (`Policy`), testes automatizados (PHPUnit/Pest), cache de respostas da PokeAPI, paginação da coleção e uma API REST própria — todos recursos que o Laravel já oferece nativamente, sem necessidade de bibliotecas externas.

> **Documento elaborado para apresentação acadêmica.**
> Stack: PHP 8.3 · Laravel 13.6 · MySQL · Blade · Tailwind CSS v4 · PokeAPI.